values as the values to match. Thus, we could have written the previous code even more succinctly:

```
pos = Order.where(params[:order])
```

Be careful with this latter form of condition: it takes *all* the key-value pairs in the hash you pass in when constructing the condition. An alternative would be to specify which parameters to use explicitly:

```
pos = Order.where(name: params[:name],
                  pay_type: params[:pay_type])
```

Regardless of which form of placeholder you use, Active Record takes great care to quote and escape the values being substituted into the SQL. Use these forms of dynamic SQL, and Active Record will keep you safe from injection attacks.

Using Like Clauses

We might be tempted to use parameterized `like` clauses in conditions:

```
# Doesn't work
User.where("name like '?%'", params[:name])
```

Rails doesn't parse the SQL inside a condition and so doesn't know that the name is being substituted into a string. As a result, it will go ahead and add extra quotes around the value of the `name` parameter. The correct way to do this is to construct the full parameter to the `like` clause and pass that parameter into the condition:

```
# Works
User.where("name like ?", params[:name]+"%")
```

Of course, if we do this, we need to consider that characters such as percent signs, should they happen to appear in the value of the name parameter, will be treated as wildcards.

Subsetting the Records Returned